



MEHRAN UNIVERSITY OF ENGINEERING AND TECHNOLOGY JAMSHORO, SINDH

SYSTEMxHACKATHON 1.0

Project Title:	QUEUEFLOW – Digital Queue and Appointment Management System
Team Name:	@Bit_Coders
Department:	Computer Systems Engineering – 24 Batch
Team Members:	Bilal Ahmed – 24CS002 Syed Muzammil Shah – 24CS016 Dileep Kumar – 24CS50 Muhammad Tarique – 24CS064



QUEUEFLOW

Digital Queue & Appointment Management System

CONTENTS

EXECUTIVE SUMMARY	- 3 -
PROBLEM STATEMENT AND PROJECT OBJECTIVES.....	- 3 -
MAIN USERS AND ROLES	- 3 -
END-TO-END SYSTEM WORKFLOW	- 4 -
SYSTEM ARCHITECTURE	- 5 -
TECHNOLOGY STACK.....	- 5 -
IMPORTANT PROJECT FOLDERS AND FILES	- 6 -
APPOINTMENT MANAGEMENT LOGIC	- 6 -
DIGITAL QUEUE AND STAFF WORKFLOW LOGIC	- 7 -
INTELLIGENT WAITING-TIME ESTIMATION	- 7 -
DATABASE AND DATA PERSISTENCE	- 7 -
SECURITY, AUTHORIZATION AND RELIABILITY	- 8 -
DASHBOARDS, REPORTS AND NOTIFICATIONS.....	- 8 -
TESTING AND VERIFICATION	- 9 -
CURRENT SCOPE AND KNOWN LIMITATIONS	- 9 -
REQUIREMENT-TO-IMPLEMENTATION SUMMARY	- 9 -
CONCLUSION.....	- 10 -
PROJECT REFERENCES	- 10 -



EXECUTIVE SUMMARY

QueueFlow is a digital queue and appointment management system designed to replace manual waiting lines, paper tokens, and disconnected appointment processes with one connected web-based workflow. The system supports scheduled appointments and walk-in visitors, tracks queue position and estimated waiting time, enables staff to call and serve customers through managed counters, and provides operational dashboards for managers and administrators.

The implemented application is a single Next.js system using the App Router, React, TypeScript, and PostgreSQL. Business-critical actions such as booking, rescheduling, token creation, check-in, calling the next customer, starting service, and completing service are handled on the server and persisted in the database. Role-based authorization separates Customer, Service Staff, Department Manager, and Administrator responsibilities.

The project emphasizes operational correctness: capacity-safe booking, unique tokens, queue fairness, compatible-counter routing, idempotent commands, historical waiting-time estimation, audit records, scheduled jobs, and automated tests. Optional AI forecasting, native mobile applications, and SMS are intentionally outside the implemented core and are not presented as completed features.

PROBLEM STATEMENT AND PROJECT OBJECTIVES

Organizations such as hospitals, government offices, banks, service centers, examination departments, and customer-care facilities often suffer from long physical queues, uncertain waiting times, missed appointments, poor counter coordination, and limited visibility for management. Manual systems make it difficult to know who should be served next, whether capacity is available, or how efficiently staff and counters are being used.

- ✚ Allow customers to book, cancel, reschedule, and check in for appointments.
- ✚ Allow walk-in visitors to obtain digital service tokens without requiring a scheduled booking.
- ✚ Combine checked-in appointments and walk-ins into one controlled mixed queue.
- ✚ Show customers their token status, people ahead, queue position, and estimated waiting time.
- ✚ Enable staff to call, recall, skip, mark missed, start, and complete customer service.
- ✚ Manage departments, services, schedules, counters, staff assignments, capacity, and closures.
- ✚ Provide role-scoped dashboards, reports, notifications, and audit history.
- ✚ Protect bookings and queue operations against duplicate or conflicting actions.

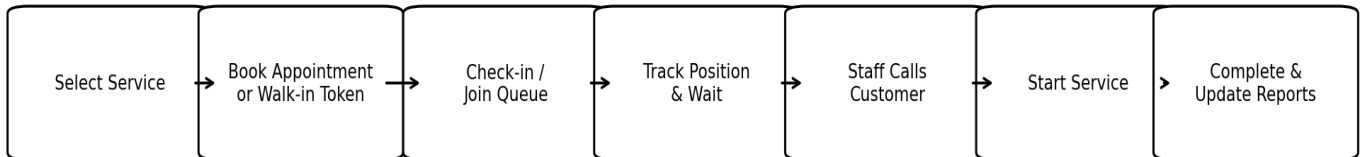
MAIN USERS AND ROLES

ROLE	MAIN RESPONSIBILITIES
Customer / Visitor	Register/login, browse departments/services, book or reschedule appointments, join a walk-in queue, check in, track waiting status, receive notifications, and view history.
Service Staff	Work from assigned counters/services, view eligible waiting customers, call



	next, recall, skip, mark missed, start service, complete service, and change counter availability.
Department Manager	Manage department-level services, schedules, capacity, staff, counters, assignments and closures; monitor workload and reports.
Administrator	Manage organization-wide departments, user accounts, services, operational configuration, activity logs, and reports.

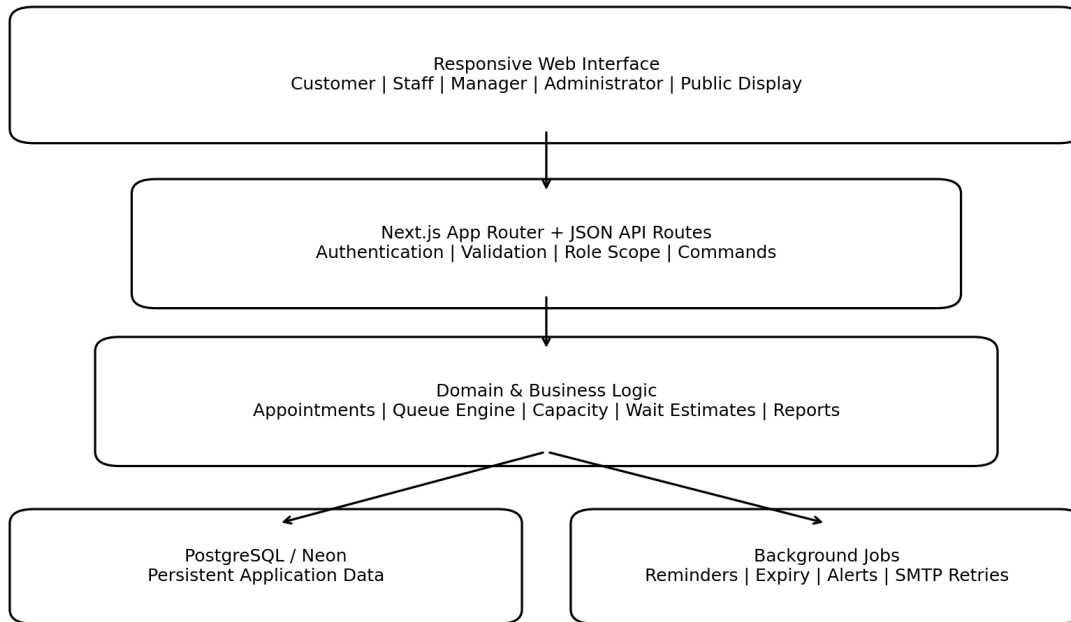
END-TO-END SYSTEM WORKFLOW



- 1) A customer chooses a department and service.
- 2) The customer either books an available appointment slot or requests a walk-in token.
- 3) For appointments, check-in is validated against the configured arrival window and one linked queue token is created.
- 4) The mixed queue engine orders eligible appointment and walk-in tokens using priority, fairness, service compatibility, and current resource state.
- 5) The customer sees current status, people ahead, and a calculated waiting estimate.
- 6) A staff member at a compatible active counter uses Call Next; the claim is performed atomically to prevent two staff members from claiming the same token.
- 7) The service is started and completed. Appointment/token history, counter state, notifications, events, and reports are updated.



SYSTEM ARCHITECTURE



The application uses a modular monolithic architecture inside one Next.js project. Browser pages and components call server-side JSON API routes. Those routes authenticate the user, validate input, enforce role/ownership scope, and invoke centralized domain logic. PostgreSQL stores authoritative data. A background worker or protected scheduled endpoint handles time-based jobs such as appointment expiry, reminders, approaching-turn alerts, cleanup, and optional email retries.

TECHNOLOGY STACK

LAYER	IMPLEMENTATION
Frontend	React with Next.js App Router; responsive role-specific interfaces.
Backend	Next.js server routes and TypeScript domain modules.
Language	TypeScript.
Database	PostgreSQL; Neon can be used as the hosted PostgreSQL provider.
Authentication	Server-side password/session handling with scoped access checks.
Background processing	Minute worker / scheduled endpoint for reminders, expiry and notifications.
Testing	TypeScript checks, domain tests, browser/end-to-end tests, production build verification.
Deployment	Vercel-compatible web deployment plus separately scheduled worker/jobs as required.



IMPORTANT PROJECT FOLDERS AND FILES

LOCATION	PURPOSE
src/app/layout.tsx, page.tsx, globals.css	Main application entry, layout and responsive styling.
src/app/display/page.tsx	Public token/counter display that avoids exposing private customer information.
src/app/api/*/route.ts	JSON API routes for authentication, commands, availability/state, display, health and scheduled operations.
src/components/workspace.tsx	Role navigation, dashboard/auth shell, notifications and five-second data refresh.
src/components/customer.tsx	Customer booking, rescheduling, check-in, active queue status and history.
src/components/operations.tsx	Staff workflow, counter operation, reports and activity log views.
src/components/management.tsx	Departments, rules, services, accounts, staff/counters and closure management.
src/lib/domain.ts	Core appointment, queue, resource, waiting-estimate, state-transition and reporting logic.
src/lib/db.ts	PostgreSQL connection pool, transactions and organization-level locking.
src/lib/auth.ts, src/lib/http.ts	Authentication, authorization, input handling, origin checks and consistent errors.
src/lib/time.ts	Organization-local date/time calculations and working-window logic.
src/lib/jobs.ts	Reminders, no-show/expiry processing, approaching-turn alerts, cleanup and SMTP retry logic.
database/*.sql	Versioned schema/migrations and historical serving-staff/duration snapshots.
scripts/	Database migration, seed and worker commands.
tests/	PostgreSQL domain scenarios and browser user journeys.
docs/	Setup, business rules, API, deployment, testing and submission documentation.

APPOINTMENT MANAGEMENT LOGIC

The main appointment rules are implemented in `src/lib/domain.ts`. Slot generation creates service-duration windows and calculates conservative shared capacity. Before reserving a slot, the system validates service availability, user overlap, per-day limits and confirmation requirements. Booking mutations run inside controlled transactions and store idempotent command results so that repeated requests do not create duplicate appointments.



- ✚ Rescheduling attempts to reserve the replacement safely and preserves the original booking if the new reservation fails.
- ✚ Cancellation releases appropriate capacity and records the change in history.
- ✚ Check-in validates the configured arrival rule and creates exactly one linked queue token.
- ✚ The linked token cannot become eligible earlier than the appointment start time.
- ✚ Starting and completing service synchronizes the queue token and related appointment status.

DIGITAL QUEUE AND STAFF WORKFLOW LOGIC

Walk-in and checked-in appointment visitors share a mixed digital queue. Token generation uses a service/day sequence to keep token references unique. The queue is not based only on token number; it considers eligibility, appointment priority, waiting age, fairness, counter/service compatibility and current counter state.

- ✚ `issueToken()` enforces active-token limits and generates service-day token sequences.
- ✚ `orderedQueue()` combines due appointments with walk-ins and raises sufficiently old walk-ins to reduce starvation.
- ✚ Call Next claims one compatible token atomically and changes the selected counter to Busy.
- ✚ Recall, Skip, Missed, Start, Complete and Cancel are controlled state transitions rather than unrestricted status edits.
- ✚ Every relevant action updates events, counter state and notification records so the operational history remains traceable.

INTELLIGENT WAITING-TIME ESTIMATION

QueueFlow includes a deterministic intelligent waiting-time estimator rather than an unverified AI model. The estimator calculates workload using the actual eligible queue order, compatible active counters, service durations, and remaining active work. Initially it uses configured/snapshotted service duration. After sufficient measured history exists, it uses the mean of recent completed-service durations (up to the recent history window documented by the project).

If there is no compatible active capacity, the system intentionally returns no numeric estimate instead of showing a misleading value. This makes the estimate operationally safer. Optional machine-learning forecasting, demand prediction, no-show prediction and queue optimization are not implemented in the current version and should not be claimed during submission.

DATABASE AND DATA PERSISTENCE

PostgreSQL is the authoritative data store. The connection is server-only through `DATABASE_URL`. The database preserves users and sessions, departments and services, counters and staff assignments, closures, appointments, tokens and token sequences, notifications, events/audit records, password reset



data, and idempotency results. Business mutations use database transactions and an organization-level lock to protect capacity allocation and queue claims across concurrent processes.

DATA GROUP	PURPOSE
Users / Sessions	Identity, roles, account state and active sessions.
Departments / Services	Service catalog, duration, rules, schedules and department ownership.
Appointments	Booking reference, date/time, service, status, check-in and rescheduling/cancellation relationships.
Tokens / Sequences	Walk-in or linked appointment token, state, service-day sequence, priority and timestamps.
Counters / Assignments	Counter state, service compatibility and staff assignment.
Notifications / Events / Audit	Operational messages, state history and traceability.
Idempotency Results	Safe replay protection for mutation commands.

SECURITY, AUTHORIZATION AND RELIABILITY

- ✚ Role and ownership checks are enforced on server operations rather than relying only on hidden UI controls.
- ✚ Customer responses are limited to the customer's own records; staff and manager operations are scoped to their permitted department/resources.
- ✚ The public display exposes safe token/counter information rather than private appointment/contact details.
- ✚ Database transactions and locking reduce race conditions in capacity reservation and token claiming.
- ✚ Idempotent mutation handling reduces duplicate bookings, tokens, check-ins or completions when requests are retried.
- ✚ Configuration and credentials are expected through environment variables; secrets should never be committed or shown in the demo video.

DASHBOARDS, REPORTS AND NOTIFICATIONS

Manager and administrator views provide operational visibility over appointments, walk-ins, waiting customers, active counters and service outcomes. Reporting includes measured waiting/service time, department and service demand, staff workload snapshots, arrival hours, approximate queue history, outcome rates and seven-day trends. Customer and public views refresh approximately every five seconds to keep the visible state current while retaining the server/database as the source of truth.

The notification subsystem supports in-application notifications and time-based events such as appointment reminders, approaching-turn alerts, counter calls, rescheduling, delays and cancellations. Email delivery is optional and requires SMTP configuration plus the worker/scheduled process.



TESTING AND VERIFICATION

The repository documents automated verification across the domain and browser layers. Domain tests run against isolated PostgreSQL environments, while browser tests exercise real user/API journeys. Additional hosted-database checks verify concurrency-sensitive scenarios such as competing bookings, reschedule rollback, exclusive token claiming and service completion through pooled connections.

VERIFICATION	PURPOSE
Type checking	Verifies TypeScript correctness.
Domain tests	Validate business rules and state transitions using PostgreSQL.
Browser / E2E tests	Exercise customer, staff, manager and administrator workflows.
Production build	Verifies the Next.js application can build for release.
Hosted PostgreSQL checks	Validate important transaction and concurrency behavior.
Dependency audit	Repository reports zero known production vulnerabilities at its recorded verification time.

CURRENT SCOPE AND KNOWN LIMITATIONS

- ✚ Live updates use periodic refresh/polling rather than a dedicated websocket infrastructure.
- ✚ Capacity allocation is intentionally conservative to avoid overbooking.
- ✚ Role permissions are fixed by the implemented role model rather than a fully dynamic permission designer.
- ✚ Calendar closures are implemented as full-day closures; partial-day overrides are not part of the current implemented scope.
- ✚ Historical hourly queue snapshots are approximate.
- ✚ Native mobile applications are not implemented.
- ✚ SMS delivery is not implemented; email is optional through SMTP configuration.
- ✚ Optional AI/ML forecasting and optimization features are not implemented.
- ✚ Manual transfer of an in-service customer between counters/services is not implemented.

REQUIREMENT-TO-IMPLEMENTATION SUMMARY

REQUIREMENT	IMPLEMENTED BEHAVIOR	MAIN LOCATION
Appointments	Booking, capacity checks, cancellation, rescheduling, check-in and history.	src/lib/domain.ts; src/components/customer.tsx; API routes
Digital Queue	Token issue, mixed ordering, fairness and state transitions.	src/lib/domain.ts; customer/staff components
Staff & Counters	Counter assignments/states and atomic Call Next workflow.	src/components/operations.tsx; src/lib/domain.ts
Waiting Estimate	Historical-duration, compatible-counter	src/lib/domain.ts (enrichTokens)



	workload calculation.	
Realtime-like Tracking	Five-second refresh with authoritative refetch.	src/components/workspace.tsx; display page
Notifications / Jobs	Reminder, expiry, approaching-turn and SMTP retry processing.	src/lib/jobs.ts
Management / Reports	Department/service/resource management, reports and activity.	management.tsx; operations.tsx; domain/report logic
Database	Persistent PostgreSQL records, migrations and transactional locking.	src/lib/db.ts; database/*.sql
Testing	Domain and browser journeys, build/type checks.	tests/; package scripts

CONCLUSION

QueueFlow delivers the complete core workflow expected from a digital queue and appointment management platform: customers can schedule or walk in, enter an eligible queue, monitor status, receive service through managed counters, and leave a persistent history that feeds operational reporting. The design keeps critical queue and booking rules on the server, uses PostgreSQL as the system of record, and protects concurrency-sensitive actions with transactions, locking and idempotent command handling.

The project is suitable for academic demonstration because its key features are connected end to end rather than being isolated screens. The strongest submission strategy is to demonstrate the real customer-to-staff lifecycle, explain where each rule is implemented, and clearly distinguish implemented historical waiting-time intelligence from optional AI extensions.

PROJECT REFERENCES

- ✚ GitHub Repository: https://github.com/tarique376/Digital-Queue-Appointment-Management-System-Bit_Coders/
- ✚ Deployed Application: <https://digital-queue-appointment-managemen.vercel.app>
- ✚ Repository README.md - implementation overview and module/requirement mapping.
- ✚ docs/PROJECT_EXPLANATION.md - authoritative implementation file map and logic summary.
- ✚ docs/SUBMISSION.md - submission package and demonstration sequence.
- ✚ docs/SETUP.md, BUSINESS_RULES.md, API.md, DEPLOYMENT.md and TESTING.md - supporting technical documentation.